

Pense-bête git



.dev et pas .com sur URL git pour ouvrir un vsCode

npm install (pour réinstaller tous les modules comme astro sur un projet)

Les plus utilisés :

git clone <https://url> : permet de créer une copie locale (clone) d'un dépôt Git distant. Elle permet de récupérer l'intégralité de l'historique du dépôt.

git pull : récupérer les dernières modifications d'un dépôt distant.

git add . : ajouter tous les fichiers modifiés, nouvellement créés ou supprimés de votre répertoire de travail à l'index de Git.

git commit -m "commit message" : enregistrer les modifications préparées dans l'index de Git dans l'historique du dépôt en créant un nouveau commit.

git push : envoyer les commits locaux vers un dépôt distant, mettant à jour ainsi l'historique du dépôt distant avec vos modifications.

git status : permet d'obtenir un statut de votre dépôt git.

git init : initialise un nouveau dépôt Git dans un répertoire local. Elle transforme un répertoire ordinaire en un dépôt Git, permettant de commencer à suivre l'historique des modifications, de créer des commits et d'utiliser d'autres fonctionnalités de Git.

git branch : connaître les branches présentes dans notre projet.

git branch [branch-name] : permet de créer une nouvelle branche.

git checkout [branch-name] : permet de se placer sur une autre branche.

git merge [destination-branch-name] : permet de fusionner (merge) la branche sur laquelle on se trouve, avec la branche de destination. Exemple : on veut fusionner une branche nommée "cagnotte" avec la branche main, on se place sur la branche main avec la commande **git checkout main** puis on vient lancer la commande **git merge cagnotte**.

git log : permet d'afficher l'historique des commits

+ possible y ajouter des options (git log --oneline, --graph, --stat, etc.).
+

git stash : permet de stocker temporairement vos modifications sur une "étagère" (stash)

git stash apply : permet de prendre ce qu'il y a sur "l'étagère" (stash) et de le mettre sur la branche sur laquelle on est.

Dépôt local et repo existants indépendamment

git init

```
git add .
```

```
git commit -m "first commit"
```

```
git remote add origin https://github.com/REPO CONCERNE
```

```
git branch -M main
```

```
git pull origin main
```

Si besoin

```
git pull origin main --allow-unrelated-histories
```

```
git push -u origin main
```

Utilisation type de Git sur un projet



- Je démarre un nouveau projet :
 - je créer un repository sur github (je n'oublie pas d'ajouter un fichier README et de choisir une licence approprié)
 - je clone le repository sur mon ordinateur de travail avec **git clone**
- Je récupère un projet déjà existant :
 - je clone directement le repository
- Je reprend le travail sur un projet que j'ai déjà cloner :
 - je mets à jour la branche sur laquelle je travaille avec **git pull**

- Je commence à travailler :
 - il existe déjà une branche pour le travail que je dois effectuer : je me place sur la branche avec **git checkout**, je la met à jour avec **git pull**, je commence à travailler.
 - Il n'existe pas de branche pour le travail que je dois effectuer : je crée la branche appropriée avec la commande **git branch**, je me place ensuite sur cette branche avec **git checkout**, je commence à travailler.

- Au cours de mon travail :
 - à chaque avancée de mon travail, je pense à commit mes modifications et apports. Pour ça, j'utilise d'abord la commande **git add .**, ensuite j'utilise la commande **git commit -m "my-message-here"**. Je prend le soin de donner un message approprié à mon commit et de l'écrire en anglais, par exemple si mon travail consistait à ajouter une image sur la page d'accueil d'un site, alors mon message ressemblera à "add an image to the homepage". Une fois que j'ai commit, j'utilise la commande **git push** pour envoyer mes modifications et apports sur le serveur distant (Github).
 - au cours de cette étape je pense à utiliser **git status** si j'ai le moindre doute sur le statut de mon dépôt git.

- J'ai finis le travail que je devais effectuer et j'ai bien pensé à effectuer un commit final :
 - je vais fusionner ma branche avec la branche principale (main) du projet, dans cet objectif je me place sur la branche principale avec la commande **git checkout**, je pense à la mettre à jour avec **git pull**
!!! mes collègues de travail on peut-être ajouter des modifications pendant que je faisais les miennes
!

- Une fois que la branche principale est à jour, je la fusionne avec ma branche à l'aide de la commande **git merge** et je gère les conflits.

git merge + nombranche qu'on veut fusionner

- git push sur la branche finale
- Si je veux retourner faire des modifs sur une branche qui existe déjà, alors j'y retourne et je merge le main dessus pour qu'elle soit à jour !

TIPS

Si pas possible de fork car en privé // que si autorisation proprio dossier
on duplique
on crée un repository vide

on clone git clone
ferme vscode

dossier éléments masqués ⇒ .git à supprimer

retour compte et je relance les lignes new repo en effaçant read me
+ add .